

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

الـ **Rate Limiter** ليس مجرد كود برمجي، بل هو الحارس الذكي الذي يضمن بقاء نظامك صامداً ومستقراً تحت أي ضغط.

Rate Limiting

ببساطة : **Rate Limiter** = نظام يحدد عدد الـ **Requests** اللي المستخدم يقدر يعملها في وقت معين

فكرته ببساطة: لو فيه مستخدم أو **Bot** يحاول يبعث **Request 1000** في الثانية، الـ **Rate Limiter** هيقفه ويقول "استنى شوية، إنت بعت **Requests** كتير" ويرجع له خطأ **429 Too Many Requests**.

ليه بنستخدم الـ **Rate Limiter** ؟ (الوظيفة)

1. منع هجمات الـ **DoS/DDoS** : بيحمي السيرفر من إنه يقع بسبب ضغط طلبات وهمية كتير.
2. تحسين الأداء : بيضمن إن مفيش مستخدم واحد "يسحب" موارد السيرفر كلها لنفسه ويخلي الموقع بطيء للباقي.
3. التحكم في التكاليف : لو بتستخدم **API** بفلوس زي **OpenAI** أو **Google Maps** ، الـ **Rate Limiter** بيخليك تتحكم في الاستهلاك عشان ما تتفاجئش بفاتورة كبيرة.

نيجي بقااا لانواع **Rate Limiter**



Concurrency Limiting (1)

يعني إيه **Concurrency Limiting**? تحديد عدد الـ **Request** اللي شغالة في نفس الوقت

مثال بسيط: تخيل: عندك **API** تقيل مثلاً بيعمل **Processing** كبير حددت: مسموح بـ **3 Requests** بس في نفس الوقت

السيناريو:

• **User1** → دخل

• **User2** → دخل

• **User3** → دخل

✗ **User4** → يستنى أو يترفض

الفكرة: بدل ما السيرفر ينهار بسبب **Requests** ثقيلة كتير في نفس الوقت بتقوله: اشتغل براحتك... بس بعدد محدود

✗ بيشتغل إزاي داخلياً؟

• بيبقى فيه عداد (**Counter**)

• كل **Request**:

◦ لو فيه مكان → يدخل

◦ لو مفيش → يستنى (**Queue**) أو يترفض

⚙ استخداماه في **.NET** بيدعم النوع ده 🕒

تفعيله: **app.UseRateLimiter();**

```
builder.Services.AddRateLimiter(options =>
{
    options.AddConcurrencyLimiter("concurrent", opt =>
    {
        opt.PermitLimit = 3; // أقصى عدد requests في نفس الوقت
        opt.QueueLimit = 2; // عدد اللي يستنوا
        opt.QueueProcessingOrder = QueueProcessingOrder.OldestFirst;
    });
});
```

```
[EnableRateLimiting("concurrent")]
public async Task<IActionResult> Get()
{
    await Task.Delay(5000); // simulate heavy work
    return Ok("Done");
}
```

استخدامه:

هـيـحـصـل إـيـه؟

- أول 3 Requests → شغالين
- اللي بعدهم: يدخل Queue (لو محدد) أو ياخذ 503 أو 429
- تستخدمه إمتى؟
- APIs ثقيلة عمليات بتأخذ وقت طويل & Database-intensive operations
- أي حاجة ممكن "تخنق" السيرفر
- Concurrency Limiting = تتحكم في "كام طلب شغال دلوقتي" مش "كام طلب في الثانية"

2 (Token Bucket)

تخيل معايا إن فيه "جردل (Bucket) محطوط عند بوابة السيرفر، والجردل ده ليه قواعد:

1. فكرة الرموز (Tokens)

- السيرفر بيرمي "كروت (Tokens)" جوه الجردل ده بمعدل ثابت (مثلاً كارت كل ثانية).
- الجردل ده ليه سعة محددة (Capacity)، يعني لو اتملى على آخره، الكروت الجديدة اللي بتنزل بتترمي بره ومش بتتحتسب.

2. عملية الطلب (Request)

- لما يجي مستخدم بيعت Request، لازم يروح للجردل ياخذ منه كارت واحد.
- لو لقي كارت: بياخده ويدخل للسيرفر والطلب يتنفذ.
- لو مالقاش كارت (الجردل فاضي): الطلب بيترفض فوراً ويرجع له 429 Too Many Requests.

ليه الـ Token Bucket عملي؟

- السر في ميزة اسمها الـ Burst الاندفاع: (تخيل إن المستخدم بقاله دقيقة مبعتش ولا طلب، فالجردل اتملى كروت (مثلاً السعة 10 كروت). فجأة المستخدم قرر بيعت 5 طلبات في نفس الثانية.
- في أنواع تانية زي Fixed Window ممكن ترفضه.



- لكن في **Token Bucket**، هياقي 10 كروت جاهزة، هياخد 5 منهم ويدخل الـ 5 طلبات كلهم "طلقة واحدة" بدون أي تأخير.

الخلاصة: هو بيسمح للمستخدم إنه يكون سريع جداً في وقت قصير (**Burst**)، بشرط إنه ما يتخطاش سعة الجردل، وبعد ما يخلص الكروت اللي معاه، هيضطر يمشي بالراحة على قد "معدل ملء الجردل" الجديد.

✳ بيشغل إزاي داخلياً؟

1. عندك **Max Tokens**

2. عندك **Refill Rate** (معدل إعادة التعبئة)

3. كل **Request** :

• لو فيه **Token → decrement**

• لو مفيش **reject →**

4. استخدامه في ASP.NET Core مثال عملي:

```
services.AddRateLimiter(rateLimiteroption =>
{
    // خلي دي برة الـ
    rateLimiteroption.RejectionStatusCode = StatusCodes.Status429TooManyRequests;

    rateLimiteroption.AddTokenBucketLimiter("Token", option =>
    {
        option.TokenLimit = 2; // مسموح بـ 2 شغالين
        option.QueueLimit = 1; // 1 يستنى في الطابور
        option.QueueProcessingOrder = QueueProcessingOrder.OldestFirst;
        option.ReplenishmentPeriod = TimeSpan.FromSeconds(30);
        option.TokensPerPeriod = 2;
        option.AutoReplenishment = true;
    })
});
```

تفعيله : **app.UseRateLimiter()**

```
[EnableRateLimiting("token")] استخدام:
public IActionResult Get()
{
    return Ok("Success");
}
```

استخدامه:

هيحصل إليه فعلياً؟

• أول 10 **Requests** → يعدوا فوراً

• بعد كده : كل ثانية يسمح بـ 2 **Requests** بس



• لو زود: يدخل Queue أو يترفض ❌

تستخدمه إمتى؟

APIs عليها ضغط عالي & Public APIs & Login / OTP بس بحذر
Systems محتاجة مرونة

⚠ نصايح مهمة:

- ما تزودش TokenLimit زيادة → هيبقى مفيش حماية
- ما تقللش قوي → هتضايق المستخدم
- استخدم Queue بحكمة

Fixed Window (3)

هو أبسط أنواع الـ Rate Limiting ، وفكرته عاملة زي "العداد اللي بيصفر نفسه" كل مدة زمنية محددة.
الفكرة الأساسية

بيتم تقسيم الوقت لـ (Windows) "زمنية ثابتة (مثلاً دقيقة واحدة). لكل نافذة، مسموح بعدد معين من الطلبات.

- لو مسموح لك بـ 5 طلبات في الدقيقة:
 - الطلبات من 1 لـ 5 هيعدوا بسلام.
 - الطلب رقم 6 في نفس الدقيقة هيترفض.
 - أول ما الدقيقة الجديدة تبدأ، العداد بيرجع لـ 0 ويسمح بـ 5 طلبات تانية.

مثال عملي بالارقام

لو ضبطت الـ Window على 1 دقيقة والـ Limit على 10 طلبات:

1. المستخدم بعث 10 طلبات في أول 10 ثواني من الدقيقة (مثلاً الساعة 1:00:10).
2. حاول بيعث طلب في الثانية رقم 30 (الساعة 1:00:30) -> هيتعمله Block
3. الساعة دقت 1:01:00 (بداية دقيقة جديدة) -> العداد صفر، والمستخدم يقدر بيعث 10 طلبات تانية.



العيب الخطير: "مشكلة التكدس (The Edge Case)"

أكبر مشكلة في الـ **Fixed Window** هي إن المستخدم "الذكي" ممكن بيعت ضعف العدد المسموح به في وقت قصير جداً لو استنى عند حدود الدقيقة.

- تخيل: مسموح بـ 10 طلبات في الدقيقة.
- المستخدم بعث 10 طلبات في آخر ثانية من الدقيقة الأولى (الساعة 1:00:59).
- وبمجرد ما الدقيقة الثانية بدأت، بعث 10 طلبات كمان في أول ثانية (الساعة 1:01:01).
- النتيجة: السيرفر استقبل 20 طلب في ثانيتين بس! وده ممكن يسبب ضغط مفاجئ (Spike) يوقع السيرفر.

• عيوبه

- ✗ مش دقيق
- ✗ ممكن يتحايل عليه بسهولة
- ✗ مش مناسب للـ APIs الحساسة

- ⚙️ في ASP.NET Core مثال عملي:

```
services.AddRateLimiter(rateLimiteroption =>
{
    rateLimiteroption.RejectionStatusCode = StatusCodes.Status429TooManyRequests;

    rateLimiteroption.AddFixedWindowLimiter("Window", option =>
    {
        option.PermitLimit = 2; // مسموح بـ 2 شغالين
        option.Window = TimeSpan.FromSeconds(600);
        option.QueueLimit = 1; // 1 يستنى في الطابور
        option.QueueProcessingOrder = QueueProcessingOrder.OldestFirst
    });
});
```

تفعيله: `app.UseRateLimiter();`

```
[EnableRateLimiting("fixed")]
public IActionResult Get()
{
    return Ok("Done");
}
```

استخدامه:



النتيجة:

- أول 5 Requests يعدوا
 - اللي بعدهم: يستنوا (Queue) أو يترفضوا (429)
- تستخدمه إمتى؟

APIs بسيطة \$ Internal systems لما مش فارق معاك الدقة العالية

⚠ امتى تتجنبه؟

Login / OTP

Public APIs كبيرة

Systems عليها ضغط عالي

Sliding Window (4)

هو النسخة "الأذكى" والمطورة من الـ Fixed Window هو موجود أساساً عشان يحل مشكلة الـ Spikes (الزحمة المفاجئة) اللي بتحصل عند حدود الدقيقة. الفكرة العبقرية: "تقسيم النافذة"

بدل ما نتعامل مع الدقيقة ككتلة واحدة صماء بتصفر نفسها كل 60 ثانية، الـ Sliding Window بيقسم الدقيقة دي لأجزاء أصغر بنسبها Segments تخيل الدقيقة (60 ثانية) متقسمة لـ 6 أجزاء، كل جزء فيه 10 ثواني:

1. العداد مش بيصفر مرة واحدة كل دقيقة.
2. كل ما يمر وقت (مثلاً 10 ثواني)، الـ "Window" بتتحرك" لقدام وتنسى أقدم 10 ثواني وتدخل مكانهم 10 ثواني جديدة.
3. الـ Limit بتاعك بيتحسب دايماً بناءً على مجموع الطلبات في الـ 6 أجزاء الأخيرة "الحالية".

ليه هو أحسن من الـ Fixed Window ؟

في الـ Fixed Window ، كان ممكن تبعت 10 طلبات في آخر ثانية و10 في أول ثانية من الدقيقة الجديدة (يعني 20 طلب في ثانيتين). في الـ Sliding Window، ده مستحيل يحصل! لأن في اللحظة اللي هتبعث

فيها الـ 10 طلبات الثانية، السيرفر هيبص على "آخر 60 ثانية" اللي هما شاملين الـ 10 طلبات اللي لسه باعتهم حالا وهيرفضك فوراً لأنك استهلكت الـ Limit بتاعك.

مثال بالأرقام (ASP.NET Core) لو عملنا الإعدادات دي:

- **PermitLimit: 100** طلب.
- **Window: 1** دقيقة.
- **SegmentsPerWindow: 6** يعني كل Segment بـ 10 ثواني.

اللي بيحصل كالتالي:

- كل 10 ثواني، السيرفر "بيرمي" أقدم Segment بالطلبات اللي كانت فيه، وبيفتح Segment جديد فاضي.
- الحسبة دايماً هي: مجموع الطلبات في الـ 6 Segments الحالية ≤ 100

-  في ASP.NET Core
- مثال:

```
services.AddRateLimiter(rateLimiteroption =>
{
    rateLimiteroption.RejectionStatusCode = StatusCodes.Status429TooManyRequests;

    rateLimiteroption.AddSlidingWindowLimiter("Window", option =>
    {
        option.PermitLimit = 2; // مسموح بـ 2 شغالين
        option.Window = TimeSpan.FromSeconds(600); // يعني 10 دقائق
        option.SegmentsPerWindow = 2; // كل قطعة مدتها 5 دقائق
        option.QueueLimit = 1; // 1 يستنى في الطابور
        option.QueueProcessingOrder = QueueProcessingOrder.OldestFirst;
    });
});
```

تفعيله : `app.UseRateLimiter();`

استخدامه:

```
[EnableRateLimiting("sliding")]
public IActionResult Get()
{
    return Ok("Done");
}
```



الكود ده "تحديد عدد الطلبات" اللي يقدر المستخدم يعملها في وقت معين علشان تمنع الضغط أو الـ **abuse** على الـ **API**

```
services.AddRateLimiter(rateLimiteroption =>
{
    rateLimiteroption.RejectionStatusCode = StatusCodes.Status429TooManyRequests;

    rateLimiteroption.AddPolicy("Policy", HttpContext =>
    RateLimitPartition.GetFixedWindowLimiter(
        partitionKey: HttpContext.Connection.RemoteIpAddress?.ToString(),
        factory: _ => new FixedWindowRateLimiterOptions
        {
            PermitLimit = 2,
            Window = TimeSpan.FromSeconds(600)
        }
    ));
}
```

partitionKey: `HttpContext.Connection.RemoteIpAddress?.ToString()`

ده أهم سطر في الكود الـ **Partition Key** هو "البصمة" اللي السيرفر بيعرف بيها مين المستخدم.

- بما إنك استخدمت الـ **IP Address**، فالسيرفر هيفتح "جردل (**Bucket**)" لكل **IP** يكلمه.
- النتيجة: لو مستخدم (أ) خلص الـ **2** طلبات بتوعه، مستخدم (ب) يقدر يدخل عادي لأن العداد بتاعه لسه 0.

يعني إيه الكلام ده عملياً؟ لو أنا دخلت من الـ **IP** بتاعي الساعة **12:00** وبعت طلبين، أي طلب ثالث هبعته لحد الساعة **12:10** هيترفض بـ **429**. أول ما الساعة تيجي **12:10**، العداد بتاعي "يصفر" وأقدر أبعت طلبين كمان.

ليه الكود ده ممتاز؟

1. حماية من الـ **Brute Force**: لو حد بيحاول يجرب باسوردات كتير من **IP** واحد، السيرفر هيقفه فوراً.
2. العدالة: مستخدم واحد "غلس" مش هيقع الموقع على باقي الناس.
3. التوفير الـ **Fixed Window**: خفيف جداً على موارد السيرفر (**Low Memory/CPU overhead**).



```
// User Limit
options.AddPolicy("userLimit", httpContext =>
    RateLimitPartition.GetFixedWindowLimiter(
        partitionKey: httpContext.User?.GetUserId(),
        factory: _ => new FixedWindowRateLimiterOptions
        {
            PermitLimit = 2,
            Window = TimeSpan.FromSeconds(20)
        }
    ));
```

هذا الكود هو "المستوى الأعلى" في تخصيص الحماية، لأنه يربط الـ **Rate Limit** بهوية المستخدم (**User Identity**) وليس بمجرد عنوان الجهاز (**IP**)

ده بيحدد: مين هو المستخدم؟" بدل ما تعتمد على: **IP** أنت هنا بتعتمد على: **UserId** من التوكن (**JWT**)
يعني إيه **Partition** ؟ كل مستخدم له عداد لوحده:

ملخص الكود: إنت هنا بتقول للسيرفر "يا سيرفر، أي حد يعمل **Login** ، افتح له عداد خاص باسمه، وممنوع بيعت أكثر من طلبين كل 20 ثانية، بغض النظر هو فاتح من كام جهاز أو من أنهي مكان في العالم".

=====

```
services.AddRateLimiter(options =>
{
    options.RejectionStatusCode = StatusCodes.Status429TooManyRequests;

    // IP Limit
    options.AddPolicy("Policy", httpContext =>
        RateLimitPartition.GetFixedWindowLimiter(
            partitionKey: httpContext.Connection.RemoteIpAddress?.ToString() ?? "unknown",
            factory: _ => new FixedWindowRateLimiterOptions
            {
                PermitLimit = 2,
                Window = TimeSpan.FromSeconds(20)
            }
        ));

    // User Limit
    options.AddPolicy("userLimit", httpContext =>
        RateLimitPartition.GetFixedWindowLimiter(
            partitionKey: httpContext.User?.GetUserId() ?? "anonymous",
            factory: _ => new FixedWindowRateLimiterOptions
            {
                PermitLimit = 2,
                Window = TimeSpan.FromSeconds(20)
            }
        ));

    // Concurrency
    options.AddConcurrencyLimiter("Concurrency", opt =>
    {
        opt.PermitLimit = 2;
        opt.QueueLimit = 2;
        opt.QueueProcessingOrder = QueueProcessingOrder.OldestFirst;
    }
    );
});

return services;
```

💡 الفكرة العامة

الكود ده بيعمل **Rate Limiting** بـ 3 طرق مختلفة:

1. حسب الـ **IP**

2. حسب المستخدم (**User**)

3. حسب عدد العمليات اللي شغالة في نفس الوقت (**Concurrency**)

✅ الخلاصة الكود ده بيعمل: تحديد عدد الطلبات لكل **IP** & تحديد عدد الطلبات لكل **User** & منع الضغط العالي على السيرفر ده **Setup** قوي جدًا وجاهز

